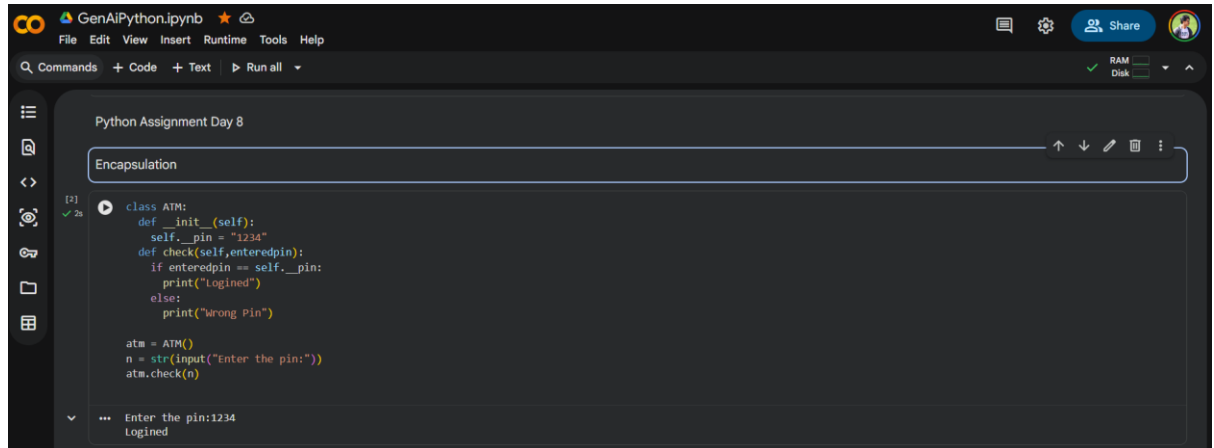


Python Assignment Day 8

1. Encapsulation



```
Python Assignment Day 8

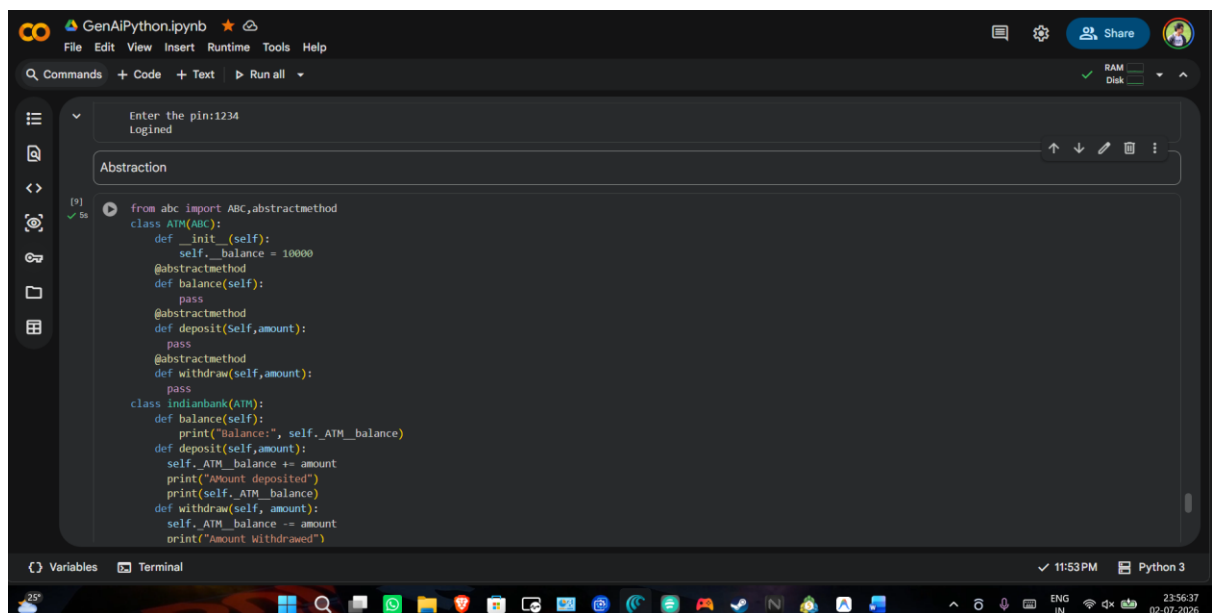
Encapsulation

class ATM:
    def __init__(self):
        self.__pin = "1234"
    def check(self, enteredpin):
        if enteredpin == self.__pin:
            print("Logged")
        else:
            print("Wrong Pin")

atm = ATM()
n = str(input("Enter the pin:"))
atm.check(n)

Enter the pin:1234
Logged
```

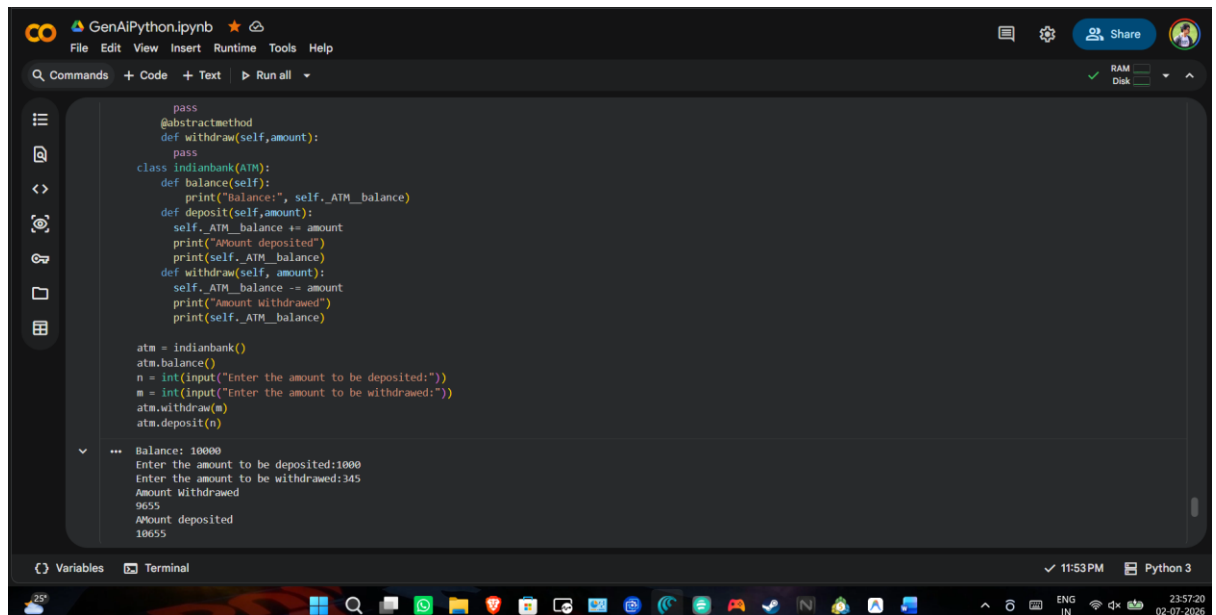
2. Abstraction



```
Enter the pin:1234
Logged

Abstraction

from abc import ABC, abstractmethod
class ATM(ABC):
    def __init__(self):
        self.__balance = 10000
    @abstractmethod
    def balance(self):
        pass
    @abstractmethod
    def deposit(self, amount):
        pass
    @abstractmethod
    def withdraw(self, amount):
        pass
class Indianbank(ATM):
    def balance(self):
        print("Balance: ", self.__balance)
    def deposit(self, amount):
        self.__balance += amount
        print("Amount deposited")
        print(self.__balance)
    def withdraw(self, amount):
        self.__balance -= amount
        print("Amount Withdrawn")
```

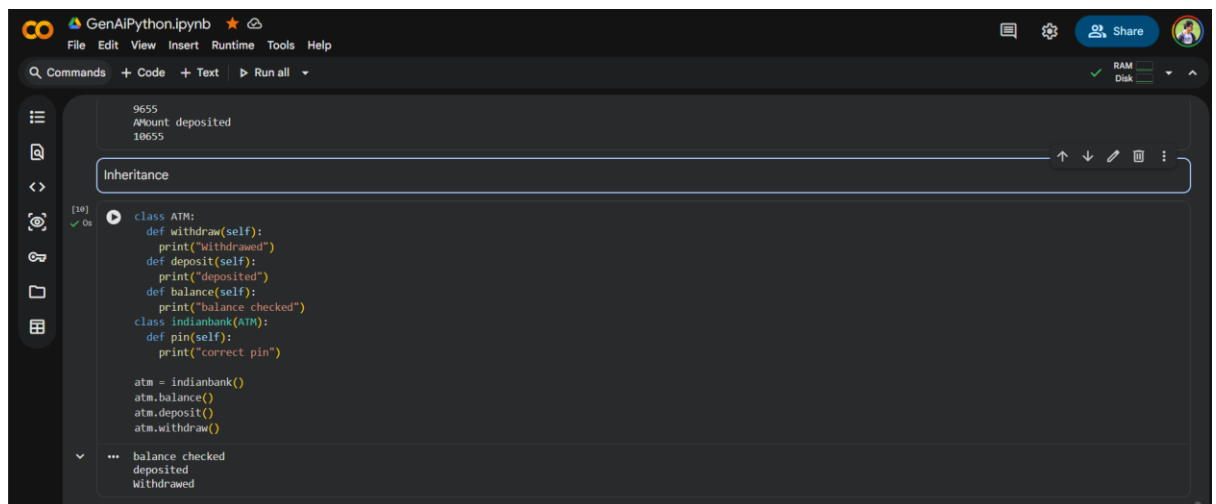


```
pass
@abstractmethod
def withdraw(self, amount):
    pass
class IndianBank(ATM):
    def balance(self):
        print("Balance:", self._ATM_balance)
    def deposit(self, amount):
        self._ATM_balance += amount
        print("Amount deposited")
        print(self._ATM_balance)
    def withdraw(self, amount):
        self._ATM_balance -= amount
        print("Amount withdrawn")
        print(self._ATM_balance)

atm = IndianBank()
atm.balance()
n = int(input("Enter the amount to be deposited:"))
m = int(input("Enter the amount to be withdrawn:"))
atm.withdraw(m)
atm.deposit(n)
```

Balance: 10000
Enter the amount to be deposited:1000
Enter the amount to be withdrawn:345
Amount Withdrawn
9655
Amount deposited
10655

3. Inheritance

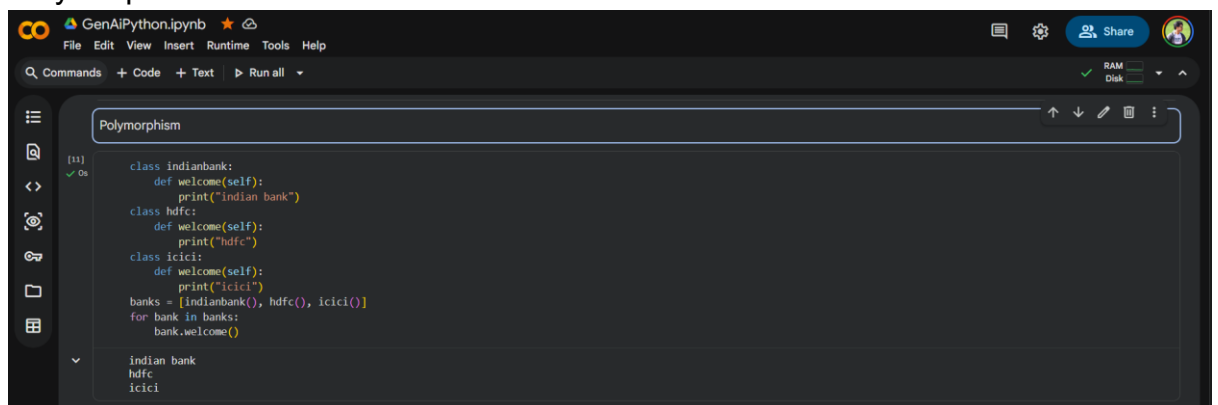


```
class ATM:
    def withdraw(self):
        print("Withdrawn")
    def deposit(self):
        print("deposited")
    def balance(self):
        print("balance checked")
class IndianBank(ATM):
    def pin(self):
        print("correct pin")

atm = IndianBank()
atm.balance()
atm.deposit()
atm.withdraw()
```

balance checked
deposited
Withdrawn

4. Polymorphism

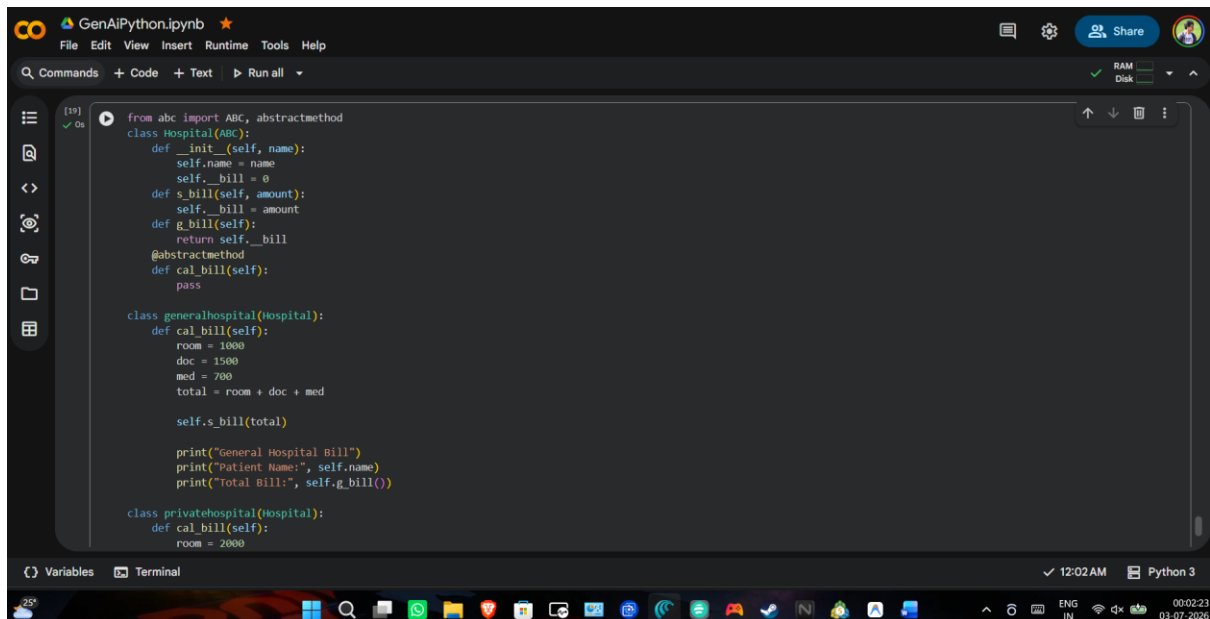


```
class IndianBank:
    def welcome(self):
        print("Indian bank")
class hdfc:
    def welcome(self):
        print("hdfc")
class icici:
    def welcome(self):
        print("icici")
banks = [IndianBank(), hdfc(), icici()]
for bank in banks:
    bank.welcome()
```

Indian bank
hdfc
icici

5. hospital billing with 4 pillars of oop,

Encapsulation, Abstraction, Inheritance, Polymorphism



```
from abc import ABC, abstractmethod

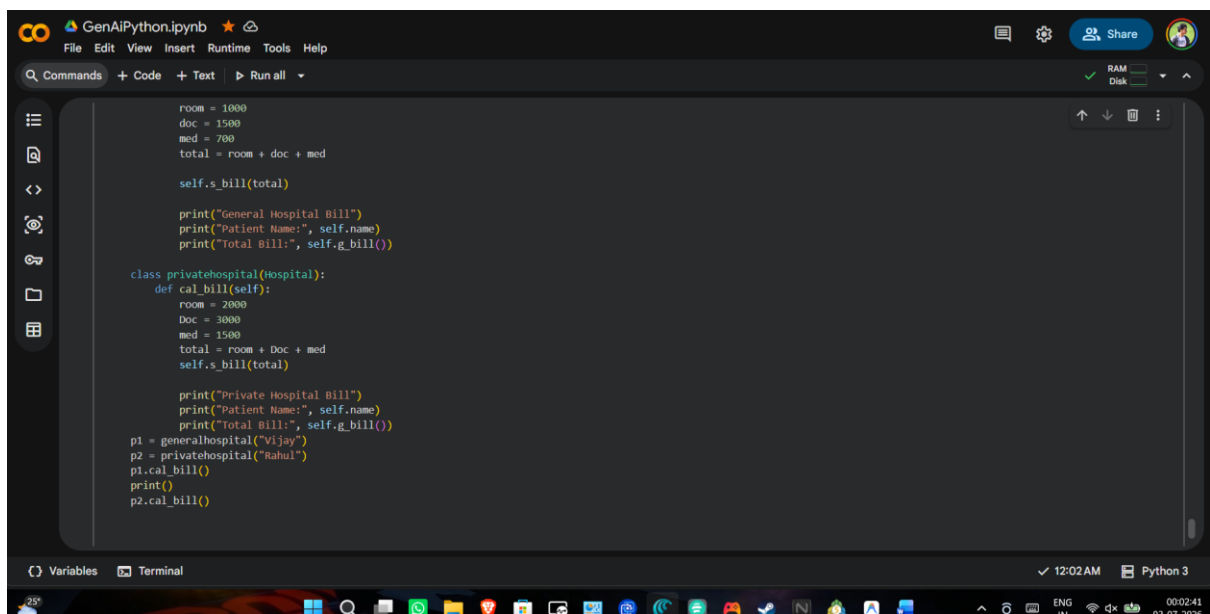
class Hospital(ABC):
    def __init__(self, name):
        self.name = name
        self._bill = 0
    def s_bill(self, amount):
        self._bill = amount
    def g_bill(self):
        return self._bill
    @abstractmethod
    def cal_bill(self):
        pass

class generalhospital(Hospital):
    def cal_bill(self):
        room = 1000
        doc = 1500
        med = 700
        total = room + doc + med

        self.s_bill(total)

        print("General Hospital Bill")
        print("Patient Name:", self.name)
        print("Total Bill:", self.g_bill())

class privatehospital(Hospital):
    def cal_bill(self):
        room = 2000
```



```
        room = 1000
        doc = 1500
        med = 700
        total = room + doc + med

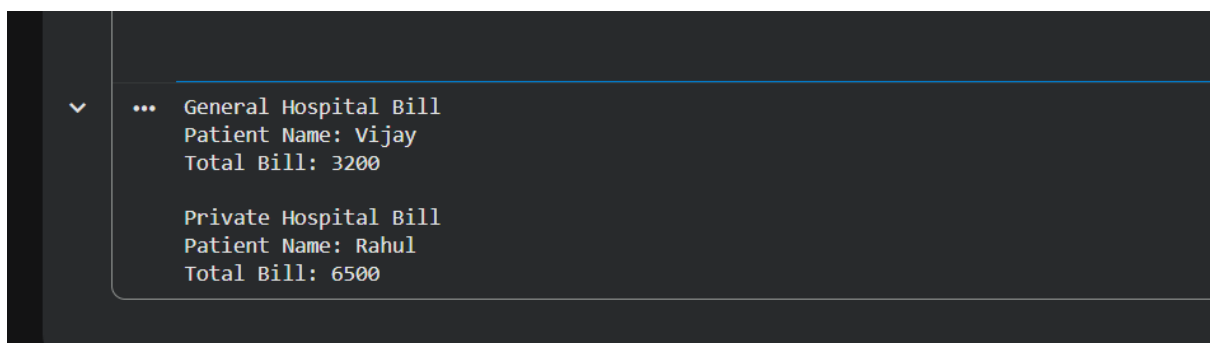
        self.s_bill(total)

        print("General Hospital Bill")
        print("Patient Name:", self.name)
        print("Total Bill:", self.g_bill())

class privatehospital(Hospital):
    def cal_bill(self):
        room = 2000
        doc = 3000
        med = 1500
        total = room + doc + med
        self.s_bill(total)

        print("Private Hospital Bill")
        print("Patient Name:", self.name)
        print("Total Bill:", self.g_bill())

p1 = generalhospital("Vijay")
p2 = privatehospital("Rahul")
p1.cal_bill()
p2.cal_bill()
print()
```



```
... General Hospital Bill
Patient Name: Vijay
Total Bill: 3200

Private Hospital Bill
Patient Name: Rahul
Total Bill: 6500
```

express encapsulation directly access the on
 class ATM:
 def __init__(self):
 self.__pin = "1234"
 def checkPin(self, enteredPin):
 if enteredPin == self.__pin:
 print("Access Granted")
 else:
 print("Wrong Pin")
 atm = ATM()
 atm.checkPin(" ")
 Abstraction
 from abc import ABC, abstractmethod
 class ATM(ABC):
 @abstractmethod
 def __init__(self):
 self.__balance = 1000
 def balance(self):
 print(self.__balance)
 pass

Date / /
 def deposit(self, amount):
 self.__balance += amount
 def withdraw(self, amount):
 self.__balance -= amount
 atm = ATM()
 from abc import ABC, abstractmethod
 class ATM(ABC):
 def __init__(self):
 self.__balance = 10000
 @abstractmethod
 def deposit(self, amount):
 pass
 @abstractmethod
 def withdraw(self, amount):
 pass

```

class Indian(ATM):
    def deposit(self, amount):
        self._balance += amount
        print("amount deposited")
        print(self._balance)

    def withdraw(self, amount):
        if amount <= self._balance:
            self._balance -= amount
            print("amount withdrawn")
            print(self._balance)
        else:
            print("insufficient balance")

atm = Indian()
atm.deposit(500)
atm.withdraw(1000)

```

Inheritance

```

class ATM:
    def balance(self):
        print("balance checked")

    def withdraw(self):
        print("balance withdrawn")

    def deposit(self):
        print("deposited")

class indian bank(ATM):
    def pin(self):
        print("correct pin")

atm = indian bank()
atm.balance()
atm.withdraw()
atm.deposit()

```

Polymorphism

```
class indianbank:
    def welcome(self):
        print("indian bank")

class hdfc:
    def welcome(self):
        print("hdfc")

class icici:
    def welcome(self):
        print("icici")

banks = [indianbank(), hdfc(), icici()]
```

for bank in banks:

bank.welcome()

5 from abc import ABC, abstractmethod

class Hospital(ABC):

def __init__(self, Name):

self.Name = Name

self.__bill = 0

def bill(self, amount):

self.__bill = amount

def get_bill(self):

return self.__bill

@abstractmethod

def calculate_bill(self):

pass

```

class generalHospital(Hospital):
    def cal_bill(self):
        room = 3000
        doc = 1500
        med = 1000
        total = room + doc + med
        self.s_bill(total)
        print("general Hospital/bill")
        print("Patient : ", self.Name)
        print("Total Bill : ", self.s_bill())

class privateHospital(Hospital):
    def cal_bill(self):
        room = 5000
        doc = 2000
        med = 2000
        total = room + doc + med
        self.s_bill(total)
        print("private Hospital/bill")
        print("Patient : ", self.Name)
        print("Total Bill : ", self.s_bill())

```

```

P1 = generalHospital("vijay")
P2 = generalHospital("sagar")
P1.cal_bill()
P1.print()
P2.cal_bill()

```